SQL Window Functions — Answer Key

Topic 1: ROW_NUMBER / RANK / DENSE_RANK

1.

```
SELECT *, ROW_NUMBER() OVER(ORDER BY marks DESC) AS row_num
FROM marks;
```

2.

```
SELECT *, RANK() OVER(ORDER BY marks DESC) AS rnk
FROM marks;
-- Ties get same rank; next rank is skipped (e.g., 2,2,4)
```

3.

```
SELECT *, DENSE_RANK() OVER(ORDER BY marks DESC) AS dense_rnk
FROM marks;
-- Ties get same rank; NO gap (e.g., 2,2,3)
```

4.

```
SELECT *,
    ROW_NUMBER()  OVER(ORDER BY marks DESC) AS row_num,
    RANK()        OVER(ORDER BY marks DESC) AS rnk,
    DENSE_RANK()  OVER(ORDER BY marks DESC) AS dense_rnk
FROM marks;
```

5.

```
SELECT *, RANK() OVER(PARTITION BY branch ORDER BY marks DESC) AS branch_rank
FROM marks;
```

6.

```
SELECT *, DENSE_RANK() OVER(PARTITION BY branch ORDER BY marks DESC) AS branch_dense_rank
FROM marks;
```

7.

```
SELECT *,
    CONCAT(branch, '-', ROW_NUMBER() OVER(PARTITION BY branch)) AS roll_number
FROM marks;
```

Topic 2: PARTITION BY with Aggregates

8.

```
SELECT *, AVG(marks) OVER(PARTITION BY branch) AS branch_avg
FROM marks;
```

9.

```
SELECT * FROM (
```

```
    SELECT *, AVG(marks) OVER(PARTITION BY branch) AS branch_avg
    FROM marks
) t
WHERE t.marks < t.branch_avg;
```

10.

```
SELECT *, MAX(marks) OVER(PARTITION BY branch) AS branch_max
FROM marks;
```

11.

```
SELECT MONTHNAME(date), user_id, SUM(amount) AS total_spent,
    RANK() OVER(PARTITION BY MONTHNAME(date) ORDER BY SUM(amount) DESC) AS monthly_rank
FROM orders
GROUP BY MONTHNAME(date), user_id
ORDER BY MONTH(date);
```

12.

```
SELECT BattingTeam, batter, SUM(batsman_run) AS total_runs,
    DENSE_RANK() OVER(PARTITION BY BattingTeam ORDER BY SUM(batsman_run) DESC) AS rank_within_tea
FROM ipl
GROUP BY BattingTeam, batter;
```

13.

```
SELECT * FROM (
    SELECT BattingTeam, batter, SUM(batsman_run) AS total_runs,
        DENSE_RANK() OVER(PARTITION BY BattingTeam ORDER BY SUM(batsman_run) DESC) AS rank_within
    FROM ipl
    GROUP BY BattingTeam, batter
) t
WHERE t.rank_within_team <= 5;
```

Topic 3: FIRST_VALUE / LAST_VALUE / NTH_VALUE

14.

```
SELECT *, FIRST_VALUE(name) OVER(ORDER BY marks DESC) AS global_topper
FROM marks;
```

15.

```
SELECT *, FIRST_VALUE(name) OVER(PARTITION BY branch ORDER BY marks DESC) AS branch_topper
FROM marks;
```

16.

```
SELECT *,
    LAST_VALUE(name) OVER(
        PARTITION BY branch ORDER BY marks DESC
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS branch_bottom
FROM marks;
```

17.

```
SELECT *,
    NTH_VALUE(name, 2) OVER(
        PARTITION BY branch ORDER BY marks DESC
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS second_topper
FROM marks;
```

18.

```
SELECT *,
    NTH_VALUE(name, 5) OVER(
        PARTITION BY branch ORDER BY marks DESC
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS fifth_topper
FROM marks;
```

19.

```
SELECT * FROM (
    SELECT *,
        FIRST_VALUE(name)  OVER(PARTITION BY branch ORDER BY marks DESC) AS topper_name,
        FIRST_VALUE(marks) OVER(PARTITION BY branch ORDER BY marks DESC) AS topper_marks
    FROM marks
) t
WHERE t.name = t.topper_name AND t.marks = t.topper_marks;
```

20.

```
SELECT name, branch, marks FROM (
    SELECT *,
        LAST_VALUE(name)  OVER w AS bottom_name,
        LAST_VALUE(marks) OVER w AS bottom_marks
    FROM marks
    WINDOW w AS (PARTITION BY branch ORDER BY marks DESC
                 ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING)
) t
WHERE t.name = t.bottom_name AND t.marks = t.bottom_marks;
```

Topic 4: Frames

21.

> Default frame when ORDER BY is present: `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`. This means each row's window only includes rows from the start of the partition up to itself — which is why running totals work naturally but LAST_VALUE doesn't return the true last row.

22.

> Without a frame, `LAST_VALUE` uses the default frame `UNBOUNDED PRECEDING TO CURRENT ROW` — so the "last" value it sees is always just the **current row itself**. Fix: add `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING` to extend the frame to the entire partition.

23.

```
SELECT *, AVG(marks) OVER(ORDER BY student_id ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING) AS moving
```

```
FROM marks;
-- Includes: the row before, the current row, and the row after
```

24.

```
-- ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING includes all rows in partition
SELECT *, SUM(marks) OVER(PARTITION BY branch ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOL
FROM marks;
```

Topic 5: WINDOW Alias

25.

```
SELECT *,
    LAST_VALUE(name)  OVER w AS bottom_name,
    LAST_VALUE(marks) OVER w AS bottom_marks
FROM marks
WINDOW w AS (PARTITION BY branch ORDER BY marks DESC
             ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING);
```

26.

```
SELECT *,
    FIRST_VALUE(name)  OVER w AS topper_name,
    FIRST_VALUE(marks) OVER w AS topper_marks
FROM marks
WINDOW w AS (PARTITION BY branch ORDER BY marks DESC);
```

Topic 6: LEAD & LAG

27.

```
SELECT *, LAG(marks) OVER(ORDER BY student_id) AS prev_marks
FROM marks;
-- First row returns NULL (no previous row)
```

28.

```
SELECT month, revenue,
    LAG(revenue) OVER(ORDER BY month) AS prev_revenue,
    ROUND((revenue - LAG(revenue) OVER(ORDER BY month)) /
          LAG(revenue) OVER(ORDER BY month) * 100, 2) AS mom_growth_pct
FROM (
    SELECT MONTH(date) AS month, SUM(amount) AS revenue
    FROM orders
    GROUP BY MONTH(date)
) monthly;
```

29.

```
SELECT *, LEAD(marks) OVER(ORDER BY student_id) AS next_marks
FROM marks;
-- Last row returns NULL (no next row)
```

Topic 7: Cumulative Sum / Average

30.

```
SELECT month, year, sales,
    SUM(sales) OVER(ORDER BY year, month
                    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS cumulative_sales
FROM monthly_sales;
```

31.

```
SELECT * FROM (
    SELECT
        CONCAT('Match-', CAST(ROW_NUMBER() OVER(ORDER BY ID) AS CHAR)) AS match_no,
        SUM(batsman_run) AS runs_scored,
        SUM(SUM(batsman_run)) OVER w AS career_runs,
        AVG(SUM(batsman_run)) OVER w AS career_avg
    FROM ipl
    WHERE batter = 'V Kohli'
    GROUP BY ID
    WINDOW w AS (ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
) t;
```

32.

```
SELECT student_id, test_number, score,
    AVG(score) OVER(PARTITION BY student_id ORDER BY test_number
                    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS cumulative_avg
FROM test_scores;
```

Topic 8: Running Average / Percent / Change

33.

```
SELECT month, year, sales,
    ROUND(AVG(sales) OVER(ORDER BY year, month
              ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 2) AS moving_avg_3
FROM monthly_sales;
```

34.

```
SELECT name,
    (votes / SUM(votes) OVER()) * 100 AS vote_pct
FROM movies;
```

35.

```
SELECT month, revenue,
    ROUND((revenue - LAG(revenue) OVER(ORDER BY month)) /
           LAG(revenue) OVER(ORDER BY month) * 100, 2) AS pct_change
FROM (
    SELECT MONTH(date) AS month, SUM(amount) AS revenue
    FROM orders
    GROUP BY MONTH(date)
) t;
```

Bonus / Mixed

36.

```
SELECT * FROM (
    SELECT BattingTeam, batter, SUM(batsman_run) AS total_runs,
        DENSE_RANK() OVER(PARTITION BY BattingTeam ORDER BY SUM(batsman_run) DESC) AS rank_withi
    FROM ipl
    GROUP BY BattingTeam, batter
) t
WHERE rank_within_team <= 3;
```

37.

```
SELECT * FROM (
    SELECT *,
        FIRST_VALUE(marks) OVER(PARTITION BY branch ORDER BY marks DESC) AS topper_marks
    FROM marks
) t
WHERE t.marks >= t.topper_marks - 10;
```

38.

```
SELECT *,
    SUM(marks) OVER(PARTITION BY branch ORDER BY student_id
                    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total
FROM marks;
```